

Lab Assignment:

Details

Weighting and due date: To be assigned by your instructor.

Submission: Include the completed lab in your lab portfolio for in-class review.

Overview

In the array-based lab, you used `ArrayList` and built an `ArrayManager`. In this lab, you will store the same kind of integer data in linked lists. Begin with Java's built-in `LinkedList<Integer>`, then build a custom singly linked list and test it through the `ConsoleProgram` framework from the review exercise.

A linked list stores values in nodes connected by references. It does not need one fixed-size array or shift later values when a node is inserted or removed. Reaching a position may still require walking through several nodes. Java's `LinkedList` is doubly linked; your custom version will use one next reference per node.

Skills Summary

- Use the built-in `LinkedList` to add, access, update, and remove values.
- Trace references through a singly linked list and handle empty and one-node lists.
- Maintain head, and count as the list changes.
- Separate data structure logic from console input and menu actions.
- Extend `ConsoleProgram` using its abstract methods, hooks, and input utilities.
- Test valid operations, invalid positions, and exception handling.

Before You Begin

Use your completed `ConsoleProgram` from the review exercise. Keep its final `run()` method, `QUIT_OPTION` constant, `readInt(String prompt)`, `startProgram()`, and `endProgram()`. The required abstract methods are `public abstract int showMenu()` and `public abstract void executeAction(int choice)`.

This lab uses lists of integers. The review exercise's random-array application is replaced by the linked-list driver described here.

Part 1 Working with LinkedList

Create a separate class named `BuiltInLinkedListDemo` with a main method. Import `java.util.LinkedList` and create a `LinkedList<Integer>` containing 12, 7, 25, 4, 18 in that order, just as you did in the previous lab.

Perform these steps in sequence. Label the output and display the list after every operation that changes it. For example:

```
System.out.println("Hello, world!");           // Hello, world!
```

1. Display the number of items
2. Display all items
3. Add item 30 to the end
4. Add item 15 at position 2
5. Remove item at position 4
6. Get item at index 4
7. Replace item at position 4
8. Add to front
9. Add to end
10. Remove first
11. Remove last

Check Your Results

After removing index 4, the list should be [12, 7, 15, 25, 18, 30]. The value at index 3 is 25. After all steps, the list should be [100, 7, 15, 25, 18, 30], with a size of 6. (**See previous lab**)

Pay Attention to Integer Removal

For `LinkedList<Integer>`, `remove(4)` removes the item at index 4. To remove the first occurrence of the value 4 instead, use `remove(Integer.valueOf(4))`. Indexes begin at 0.

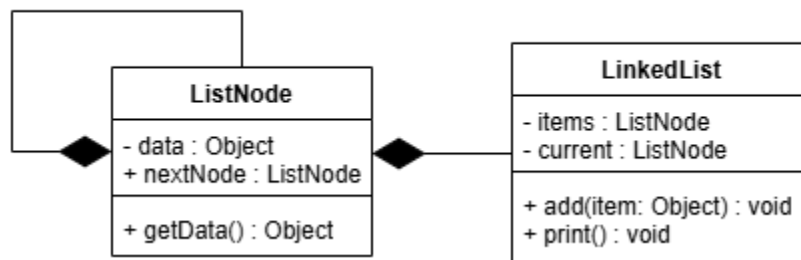
Think About Traversal

Use an enhanced for loop when displaying the built-in list. Repeated `get(i)` calls in an index-based loop require repeated traversal and can make a full display take quadratic time, defeating the purpose of using this structure.

Part 2 Building a custom LinkedList

Create LinkedList to store int values in a custom singly linked list. Do not use an array or a Java collection as the internal storage. An input array is allowed only in the constructor that copies initial values into new nodes.

A linked list is made up of classes that contain a reference to its own type. The following class diagram depicts a simplified linked list. (Constructors and all get/set methods not shown for brevity but should always be included as a best practice).



The list node class has some data and a reference to the next item in the list. It looks like this:

```

public class ListNode{
    private Object data;
    public ListNode nextNode;

    public ListNode(Object item){
        data=item;
    }

    public Object getData(){
        return data;
    }
}
  
```

The linked list class manages a collection of connected **ListNode**s, and provides operations of manipulating them. Here we provide a simplified add (it only adds to the end of the list), and print.

```
public class LinkedList {

    //A link to the first item in the linked list
    private ListNode items;

    //The current item (in this case always the last item)
    private ListNode current;

    // Add checks to see if this is the first item added.
    //     If it is, it assigns it to items
    //     If it's not, it assigns it after current, then makes item the
    //     current
    public void add (Object item) {
        if (items == null) {
            items = new ListNode(item);
            current=items;
        }
        else {
            current.nextNode = new ListNode(item);
            current = current.nextNode;
        }
    }

    // Print provides us with our basic traversal algorithm.
    // This is like the for loop for pointer structures.
    // If there is a node, print it, and advance to the next node.
    public void print(){
        ListNode temp = items;

        while (temp != null){
            System.out.println(temp.getData());
            temp = temp.nextNode;
        }
    }
}
```

A simple method in your extended console program to test your linked list:

```
public static void runLinkedListTest (){
    LinkedList list = new LinkedList();

    list.add(5);
    list.add(6);
    list.add(7);
    list.add(8);
    list.add(9);
    list.add(10);
    list.add(11);
    list.add(12);

    list.print();
}
```

Part 3 Enhancing the Custom LinkedList

Modify the linked list class so it includes the following methods and matches our new UML:

+start(): void

- Starts traversal by setting the current parser equal to the start of the list

+addAfter(Object item): void

- Adds item after current
- If there are no items in the list, it makes item the first ListNode in the list

+getCurrent(): Object

- Returns the current ListNode's data

+advance(): boolean

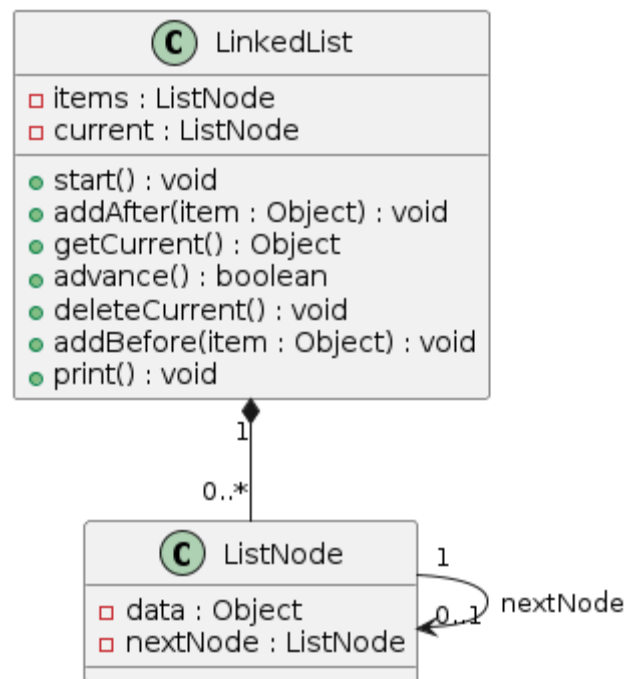
- Advances current parser and returns true if successful, or returns false if the current node is the last item in the array

+deleteCurrent(): void

- Deletes the current node from the list and makes the next node the current one

+addBefore(Object item): void

- Adds item before the current item
- If there are no items in the list, it makes item the first ListNode in the list



Note: You can delete the original Add method from the previous exercise and replace its use with `addBefore` or `addAfter`, depending on the context.

Driver: Write a driver, based on our console framework that allows the user to manipulate a linked list. Include the following menu options:

```
===== MENU =====  
1. Add Before          4. Get Current          7. Advance  
2. Add After           5. Get Item At          8. Start  
3. Print               6. Delete Current       9. Run Test  
0. Exit Program
```

What to Submit

Ensure everything in your GitHub repository is committed & up to date. Indicate the commit in Brightspace if your submission is different than the most recent push to your main branch.

Additional Resources

→ [PlantUML](#) (try it yourself!)

```
@startuml  
class ListNode {  
    - data : Object  
    - nextNode : ListNode  
}  
  
class LinkedList {  
    - items : ListNode  
    - current : ListNode  
  
    + start() : void  
    + addAfter(item : Object) : void  
    + getCurrent() : Object  
    + advance() : boolean  
    + deleteCurrent() : void  
    + addBefore(item : Object) : void  
    + print() : void  
}  
  
LinkedList "1" *-- "0..*" ListNode  
ListNode "1" --> "0..1" ListNode : nextNode  
@enduml
```